

Predicting TikTok 30-Day View Counts

Forecasting day-30 reach from the first five days · WeCloudData DS Bootcamp

The task

Predict the cumulative view count a video will have reached 30 days after posting, using only what is known in its first five days. Scored by RMSE.

The data

table	rows	what it holds
videos	209,543	metadata, topic, music, duration, emotion scores
creator_daily	278,433	daily follower and favourite counts
engagement_daily	6,068,955	daily cumulative views, likes, shares

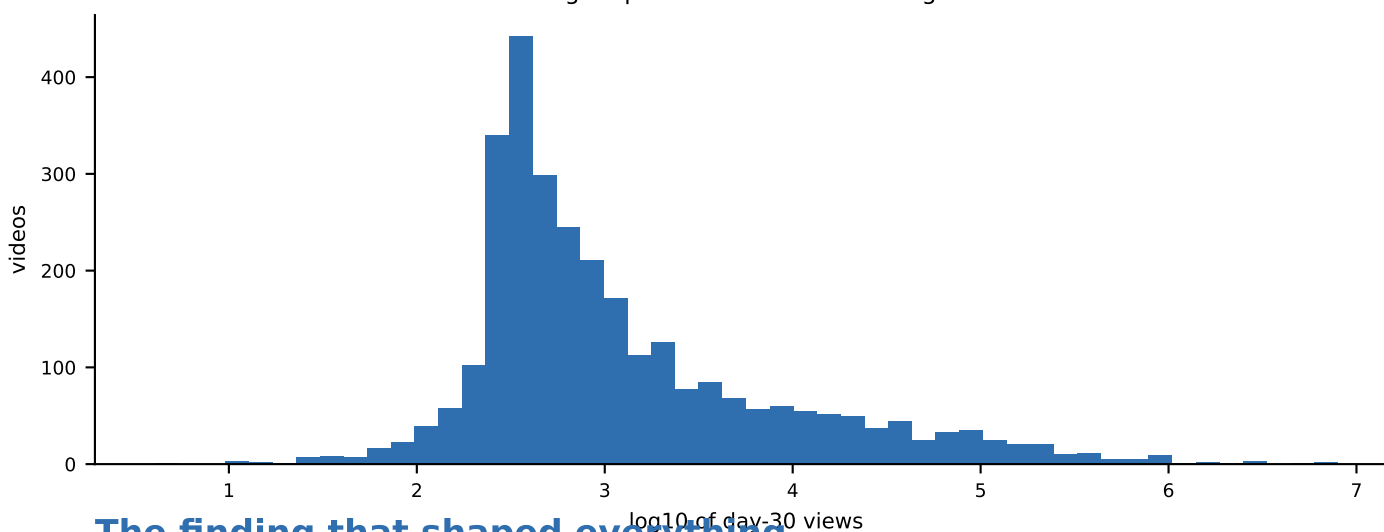
Only 73,940 videos have a day-30 observation, so those are the modelling universe. 3,001 were held out and never touched during training. Two columns were dropped at once: `is_ads` has a single value, and `enterprise_verified` is entirely null.

Why this problem is hard

Reach follows a power law. Half the videos never pass 670 views while the largest reaches 20.7 million — a skew of 30. Because RMSE squares the error, one miss on a 5-million-view video outweighs thousands of misses on 400-view videos.

p50	p90	p99	max	skew
670	21,598	617,106	20,653,259	30.4

The target spans seven orders of magnitude



The finding that shaped everything

Before tuning anything I measured where the error actually lives. The top decile of videos accounts for essentially 100% of the squared error, and the top 1% alone accounts for 97.7%. The bottom 90% contributes nothing measurable. This competition is decided by a few hundred videos, so every decision below optimises for them.

Approach

1. Predict the growth ratio, not the level

Views are cumulative — 99.4% of series never decrease — so day-30 equals day-5 multiplied by some growth factor. That multiplier has a median of 1.16 and almost no skew, while the level spans seven orders of magnitude. So instead of predicting the number, the model predicts $\log_{10}(y)$ minus $\log_{10}(\text{day-5 views})$, then adds the anchor back.

This single change was worth 25,700 RMSE — the largest gain in the whole project. It came from understanding how the data is recorded, not from a bigger model.

2. Weight the fit by y squared

The log-ratio fit treats a 300-view video and a 3-million-view video as equally important, which is wrong for RMSE. In log space an error e becomes a raw error of roughly y times e , so weighting each row by y squared makes the log-space fit approximate raw-scale least squares. Worth a further 6,100 RMSE.

3. Blend five models

Three ratio models at different weightings, plus a raw-L2 model and a Tweedie model. Weights come from non-negative least squares on out-of-fold predictions. A paired bootstrap confirmed the blend beats every individual model with the 95% interval excluding zero — the ratio models and the level models fail on different videos.

4. Features — 319, all from days 0 to 5 only

- Trajectory: levels, daily increments, acceleration, growth multipliers, log-slope and curvature for each of 7 metrics
- Ratios: shares, likes and comments per view at day 1 and day 5, and the change between them — a rising share rate is the classic pre-viral signal
- Peer context: this video against its creator's, its sound's and its topic's usual performance, plus percentile ranks
- Creator stats as of the posting date, joined with merge_asof direction='backward' so no future information can leak in

Progress

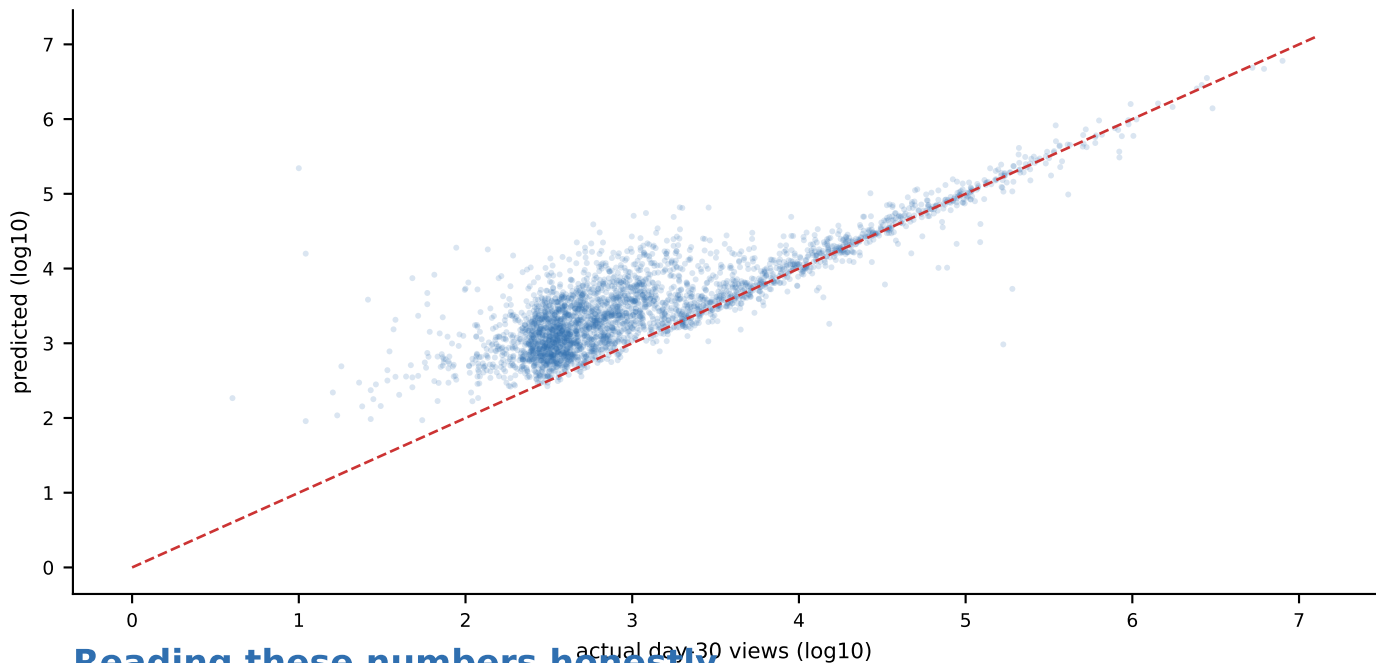
step	CV RMSE
Predict the mean	324,033
Day-5 views as-is	161,605
LightGBM on the raw target	152,860
Log target	157,999
Growth ratio vs day 5	127,159
+ weight by y squared	121,069
+ peer-context features	119,823
5-model blend	114,748

Results

3,001 held-out videos, never seen during training

metric	value	reading
RMSE	62,374	the competition metric
vs predict-the-mean	243,652	74% better
vs day-5 baseline	129,905	52% better
R squared	93.4%	variance explained
Spearman	77.9%	rank agreement
Top 1% median error	16.6%	87% within 2x

Holdout: predicted vs actual



Reading these numbers honestly

R squared of 93.4% is the headline, but the model is deliberately poor on small videos — its median error across all videos is about 150%. That is not a defect. A 400-view video predicted at 1,000 contributes nothing to RMSE, while the top 1% is predicted to within 17% and lands inside a factor of two 87% of the time. The model spends its capacity where the metric does.

If the competition were scored on percentage error instead, the right design would be different: a plain log-target model with no tail weighting, which scored worse on RMSE but far better on median error. The metric drives the design.

What did not work

Kept deliberately — the failures explain the design better than the successes do.

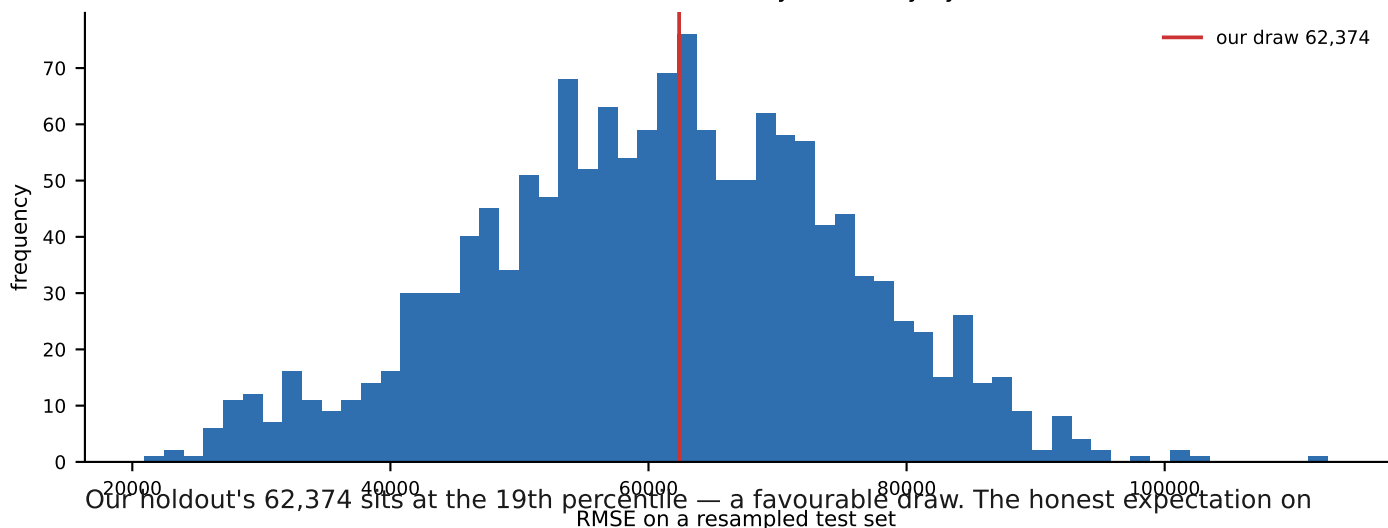
attempt	result	why
Creator history encoding	much worse 134k vs 120k	virality is video-specific, not creator-specific; past growth does not transfer
Duan smearing	worse	ratio form already removes most bias
Exact raw-L2 objective	diverged	exp(2z) Hessian explodes on 7-figure targets
Tail-specialist model	worse	training on 10% of rows loses more than it gains
Capping sample weights	worse	capped videos are the ones RMSE cares about
Segment-wise blending	inside noise	smaller than fold-to-fold variation

The limitation that matters most

The score is dominated by luck. Scoring the identical model on 2,000 different random 3,001-video test sets produces this spread:

p5	p25	p50	p75	p95
48,346	67,111	85,637	120,464	223,322

The same model scores very differently by draw



Underneath this sits a hard limit: 13 videos out of 73,922 carry 37% of all squared error. They were small at day 5 and exploded afterwards. Nothing in the first five days distinguishes them, so that portion of the error is irreducible with this data.

Files

build_features.py, add_features.py, final.py, ensemble.py, compare.py, diagnose.py, tiktok_day30_views.ipynb, submissions/submission.csv